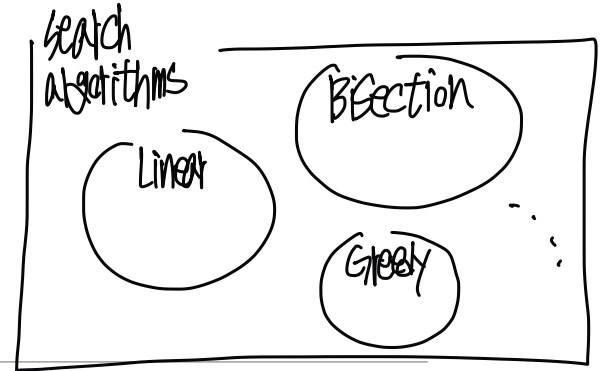


SEARCHING AND SORTING ALGORITHMS

(download slides and .py files and follow along!)

6.0001 LECTURE 12

SEARCH ALGORITHMS



- search algorithm – method for finding an item or group of items with specific properties within a collection of items
- collection could be implicit
 - example – find square root as a search problem
 - exhaustive enumeration
 - bisection search
 - Newton-Raphson
- collection could be explicit
 - example – is a student record in a stored collection of data?

SEARCHING ALGORITHMS

■ linear search

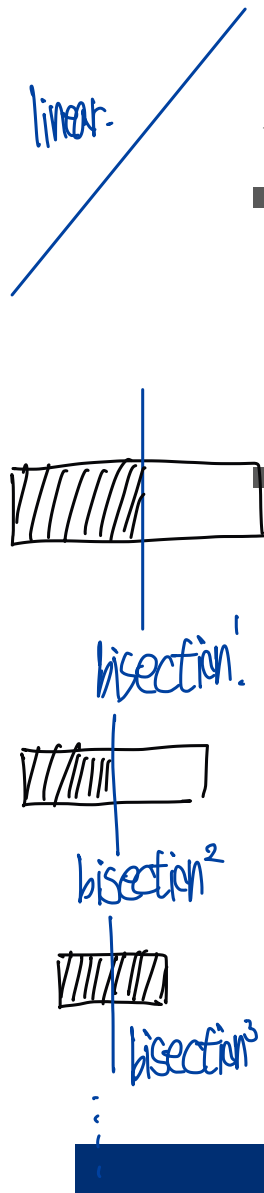
- **brute force** search (aka British Museum algorithm)
- list does not have to be sorted

■ bisection search

- list **MUST be sorted** to give correct answer
- saw two different implementations of the algorithm

[1, 2, 3, 4, 5] (correct)

[1, 3, 2, 7, 6] (correct x)



LINEAR SEARCH ON **UNSORTED** LIST: RECAP

⇒ linear search of linear search or

```
def linear_search(L, e):  
    found = False  
    for i in range(len(L)):  
        if e == L[i]:  
            found = True  
    return found
```

listname.sort() → binary search.

*Recommend X
O(n) Big O ↑*

*speed up a little by
returning True here,
but speed up doesn't
impact worst case*

- must look through all elements to decide it's not there
- $O(\text{len}(L))$ for the loop * $O(1)$ to test if $e == L[i]$
- overall complexity is **$O(n)$ – where n is $\text{len}(L)$**

*Assumes we can
retrieve element
of list in constant
time*

LINEAR SEARCH ON **SORTED** LIST: RECAP

⇒ Can Bisection search OR. Search.

```
def search(L, e):  
    for i in range(len(L)):  
        if L[i] == e:  
            return True  
        if L[i] > e:  
            return False  
    return False
```

- must only look until reach a number greater than e
- $O(\text{len}(L))$ for the loop * $O(1)$ to test if $e == L[i]$
- overall complexity is **$O(n)$ – where n is $\text{len}(L)$**

USE BISECTION SEARCH: RECAP

1. Pick an index, i , that divides list in half
2. Ask if $L[i] == e$
3. If not, ask if $L[i]$ is larger or smaller than e
4. Depending on answer, search left or right half of L for e

A new version of a divide-and-conquer algorithm

- Break into smaller version of problem (smaller list), plus some simple operations
- Answer to smaller version is answer to original problem

BISECTION SEARCH

IMPLEMENTATION: RECAP

```
def bisect_search2(L, e):
    def bisect_search_helper(L, e, low, high):
        if high == low:
            return L[low] == e
        mid = (low + high)//2
        if L[mid] == e:
            return True
        elif L[mid] > e:
            if low == mid: #nothing left to search
                return False
            else:
                return bisect_search_helper(L, e, low, mid - 1)
        else:
            return bisect_search_helper(L, e, mid + 1, high)
    if len(L) == 0:
        return False
    else:
        return bisect_search_helper(L, e, 0, len(L) - 1)
```

COMPLEXITY OF BISECTION SEARCH: RECAP

- **bisect_search2** and its helper
 - $O(\log n)$ bisection search calls
 - reduce size of problem by factor of 2 on each step
 - pass list and indices as parameters
 - list never copied, just re-passed as pointer
 - constant work inside function
 - $\rightarrow O(\log n)$

SEARCHING A SORTED LIST

-- n is $\text{len}(L)$

- using **linear search**, search for an element is **$O(n)$**
- using **binary search**, can search for an element in **$O(\log n)$**
 - assumes the **list is sorted!**
- when does it make sense to **sort first then search?**
 - $\text{SORT} + O(\log n) < O(n) \rightarrow \text{SORT} < O(n) - O(\log n)$
 - when sorting is less than $O(n)$
- **NEVER TRUE!**
 - to sort a collection of n elements must look at each one at least once!

AMORTIZED COST

-- n is $\text{len}(L)$

- why bother sorting first?
- in some cases, may **sort a list once** then do **many searches**
- **AMORTIZE cost** of the sort over many searches
- $\text{SORT} + K * O(\log n) < K * O(n)$
 - for large K , **SORT time becomes irrelevant**, if cost of sorting is small enough

SORT ALGORITHMS

- Want to efficiently sort a list of entries (typically numbers)
- Will see a range of methods, including one that is quite efficient

MONKEY SORT

- aka bogosort, stupid sort, slowsort, permutation sort, shotgun sort
- to sort a deck of cards
 - throw them in the air
 - pick them up
 - are they sorted?
 - repeat if not sorted



COMPLEXITY OF BOGO SORT

```
def bogo_sort(L):  
    while not is_sorted(L):  
        random.shuffle(L)
```

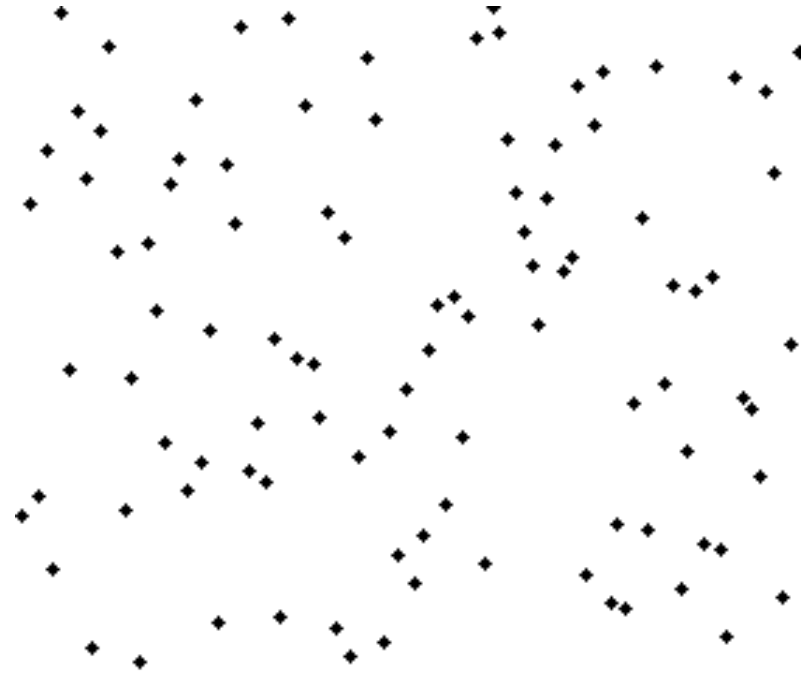
- best case: **$O(n)$** where n is $\text{len}(L)$ to check if sorted
- worst case: $O(?)$ it is **unbounded** if really unlucky

BUBBLE SORT

- **compare consecutive pairs** of elements
- **swap elements** in pair such that smaller is first
- when reach end of list, **start over** again
- stop when **no more swaps** have been made
- largest unsorted element always at end after pass, so at most n passes

CC-BY Hydrargyrum

https://commons.wikimedia.org/wiki/File:Bubble_sort_animation.gif



COMPLEXITY OF BUBBLE SORT

```
def bubble_sort(L):  
    swap = False  
    while not swap: O(len(L))  
        swap = True  
        for j in range(1, len(L)): O(len(L))  
            if L[j-1] > L[j]:  
                swap = False  
                temp = L[j]  
                L[j] = L[j-1]  
                L[j-1] = temp
```

- inner for loop is for doing the **comparisons**
- outer while loop is for doing **multiple passes** until no more swaps
- **$O(n^2)$ where n is $\text{len}(L)$**
to do $\text{len}(L)-1$ comparisons and $\text{len}(L)-1$ passes

SELECTION SORT

- first step
 - extract **minimum element**
 - **swap it** with element at **index 0**
- subsequent step
 - in remaining sublist, extract **minimum element**
 - **swap it** with the element at **index 1**
- keep the left portion of the list sorted
 - at i 'th step, **first i elements in list are sorted**
 - all other elements are bigger than first i elements

ANALYZING SELECTION SORT

- loop invariant
 - given prefix of list $L[0:i]$ and suffix $L[i+1:\text{len}(L)]$, then prefix is sorted and no element in prefix is larger than smallest element in suffix
 1. base case: prefix empty, suffix whole list – invariant true
 2. induction step: move minimum element from suffix to end of prefix. Since invariant true before move, prefix sorted after append
 3. when exit, prefix is entire list, suffix empty, so sorted

COMPLEXITY OF SELECTION SORT

```
def selection_sort(L):  
    suffixSt = 0  
    while suffixSt != len(L):  
        for i in range(suffixSt, len(L)):  
            if L[i] < L[suffixSt]:  
                L[suffixSt], L[i] = L[i], L[suffixSt]  
        suffixSt += 1
```

*len(L) times
→ $O(\text{len}(L))$*

*len(L) - suffixSt times
→ $O(\text{len}(L))$*

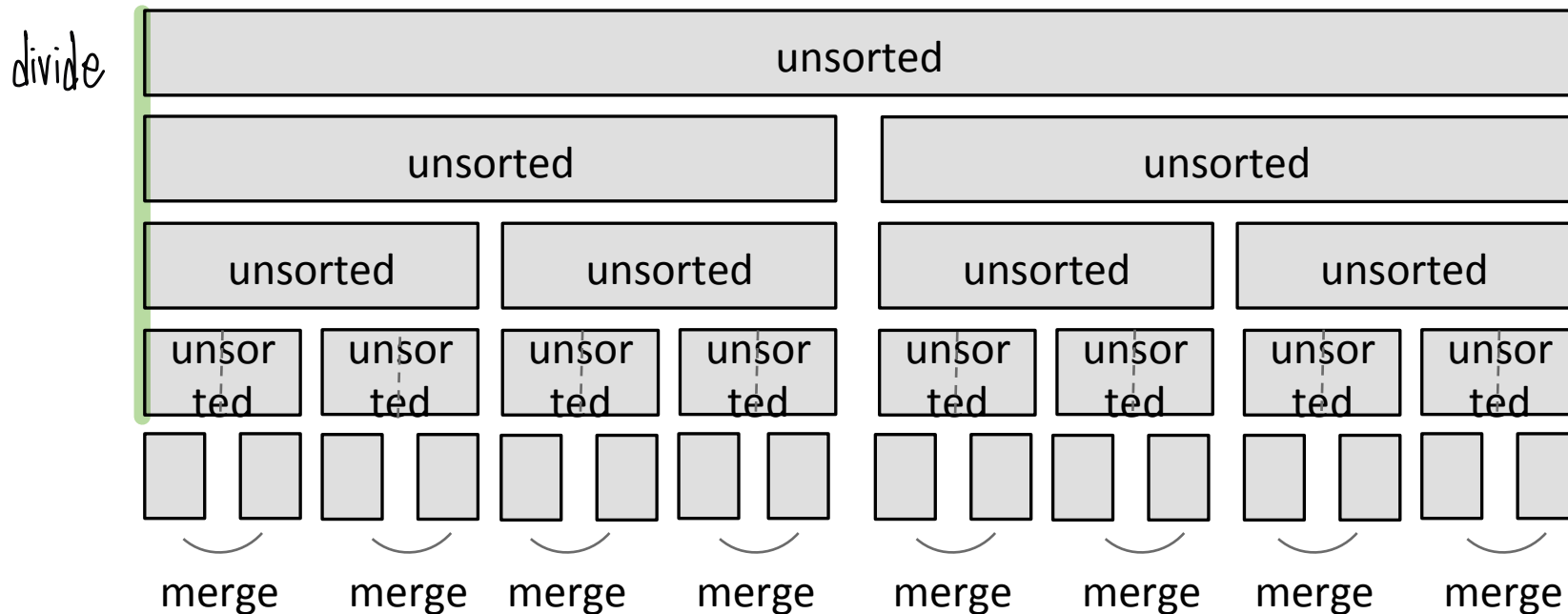
- outer loop executes $\text{len}(L)$ times
- inner loop executes $\text{len}(L) - i$ times
- complexity of selection sort is **$O(n^2)$ where n is $\text{len}(L)$**

MERGE SORT

- use a divide-and-conquer approach:
 1. if list is of length 0 or 1, already sorted
 2. if list has more than one element, split into two lists, and sort each
 3. merge sorted sublists
 1. look at first element of each, move smaller to end of the result
 2. when one list empty, just copy rest of other list

MERGE SORT

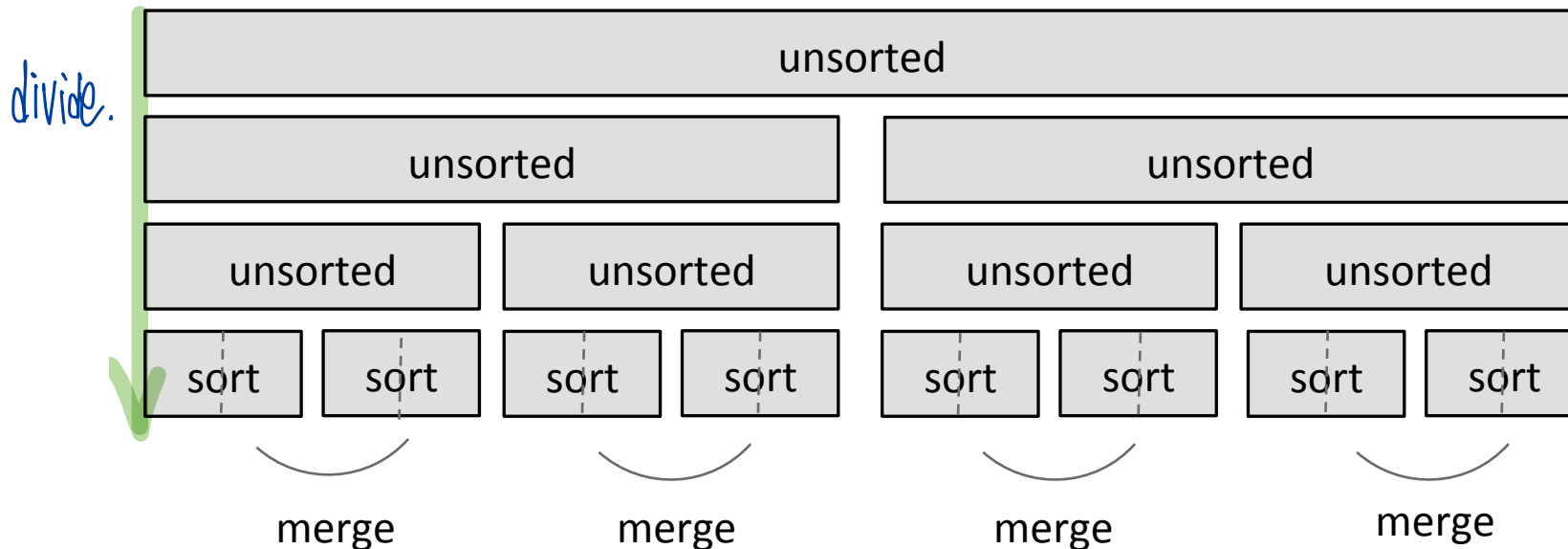
- divide and conquer



- **split list in half** until have sublists of only 1 element

MERGE SORT

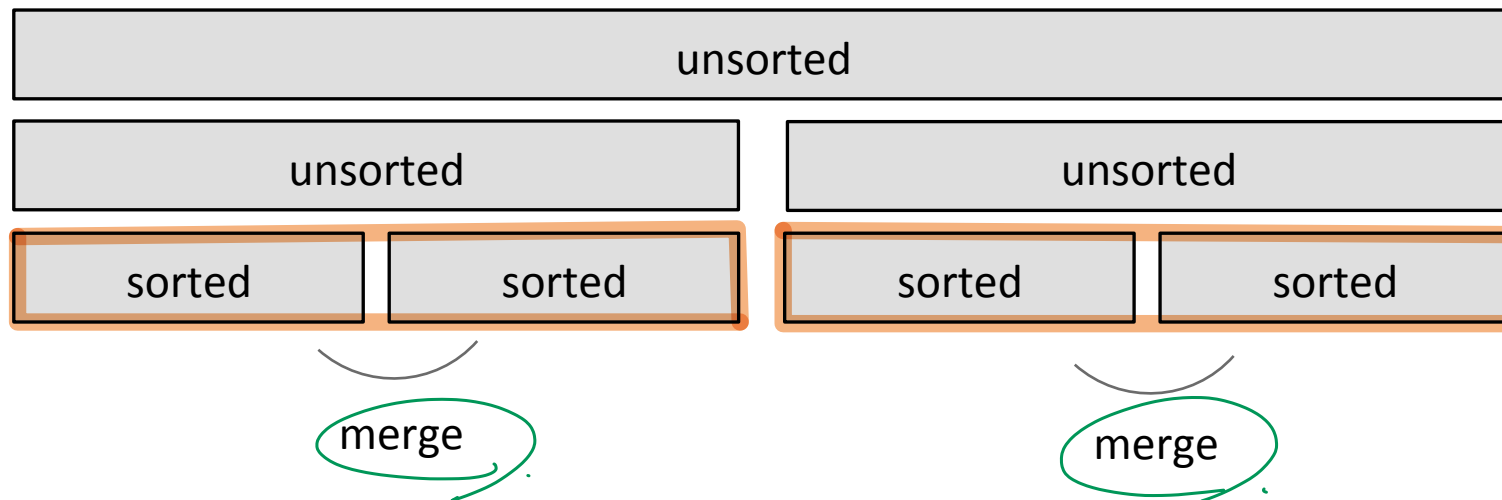
- divide and conquer



- merge such that **sublists will be sorted after merge**

MERGE SORT

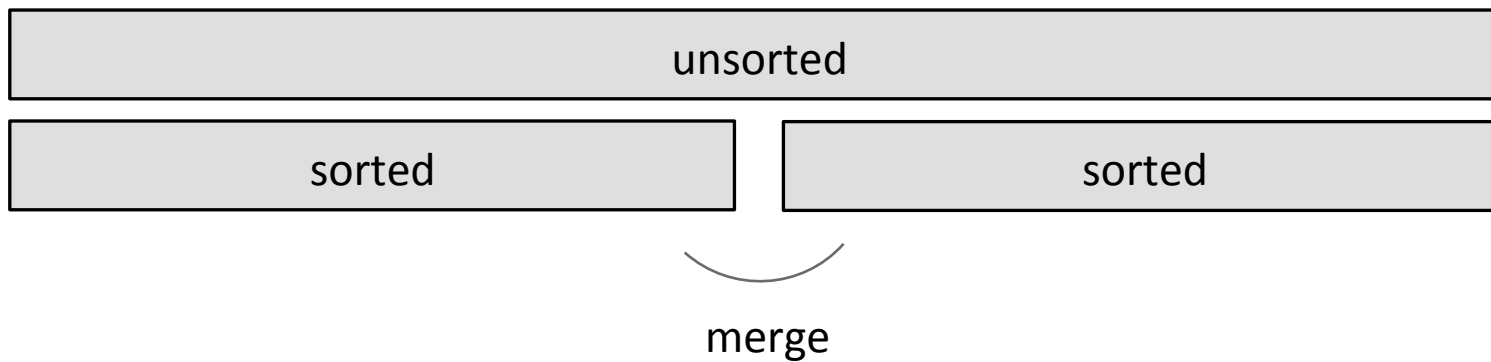
- divide and conquer



- merge sorted sublists
- sublists will be sorted after merge

MERGE SORT

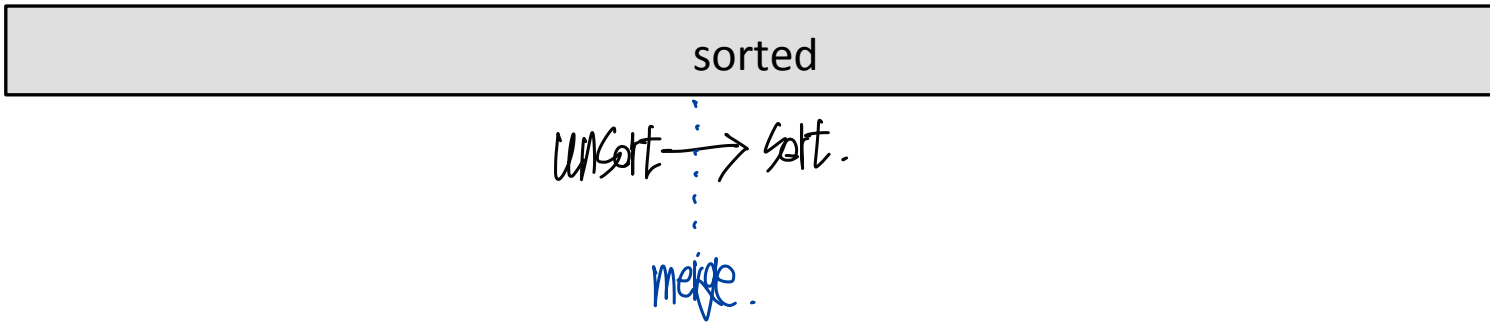
- divide and conquer



- merge sorted sublists
- sublists will be sorted after merge

MERGE SORT

- divide and conquer – done!



EXAMPLE OF MERGING

Left in list 1	Left in list 2	Compare	Result
[<u>1</u> 5,12,18,19,20]	[<u>2</u> 3,4,17]	<u>1</u> , 2 →	[<u>1</u>]
[<u>5</u> ,12,18,19,20]	[<u>2</u> 3,4,17]	5, <u>2</u> →	[1, <u>2</u>]
[<u>5</u> ,12,18,19,20]	[<u>3</u> 4,17]	5, <u>3</u> →	[1,2, <u>3</u>]
[5,12,18,19,20]	[4,17]	5, 4	[1,2,3,4]
[5,12,18,19,20]	[17]	5, 17	[1,2,3,4,5]
[12,18,19,20]	[17]	12, 17	[1,2,3,4,5,12]
[18,19,20]	[17]	18, 17	[1,2,3,4,5,12,17]
[18,19,20]	[]	18, --	[1,2,3,4,5,12,17,18]
[]	[]		[1,2,3,4,5,12,17,18,19,20]

MERGING SUBLISTS STEP

```
def merge(left, right):  
    result = []  
    i, j = 0, 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    while (i < len(left)):  
        result.append(left[i])  
        i += 1  
    while (j < len(right)):  
        result.append(right[j])  
        j += 1  
    return result
```

- left and right sublists
are ordered
- move indices for
sublists depending on
which sublist holds next
smallest element

when right
sublist is empty

when left
sublist is empty

COMPLEXITY OF MERGING SUBLISTS STEP

- go through two lists, only one pass
- compare only **smallest elements in each sublist**
- $O(\text{len}(\text{left}) + \text{len}(\text{right}))$ copied elements
- $O(\text{len}(\text{longer list}))$ comparisons
- **linear in length of the lists**

MERGE SORT ALGORITHM

-- RECURSIVE

```
def merge_sort(L):
```

```
    if len(L) < 2:  
        return L[:]
```

```
    else:
```

```
        middle = len(L)//2
```

```
        left = merge_sort(L[:middle])
```

```
        right = merge_sort(L[middle:])
```

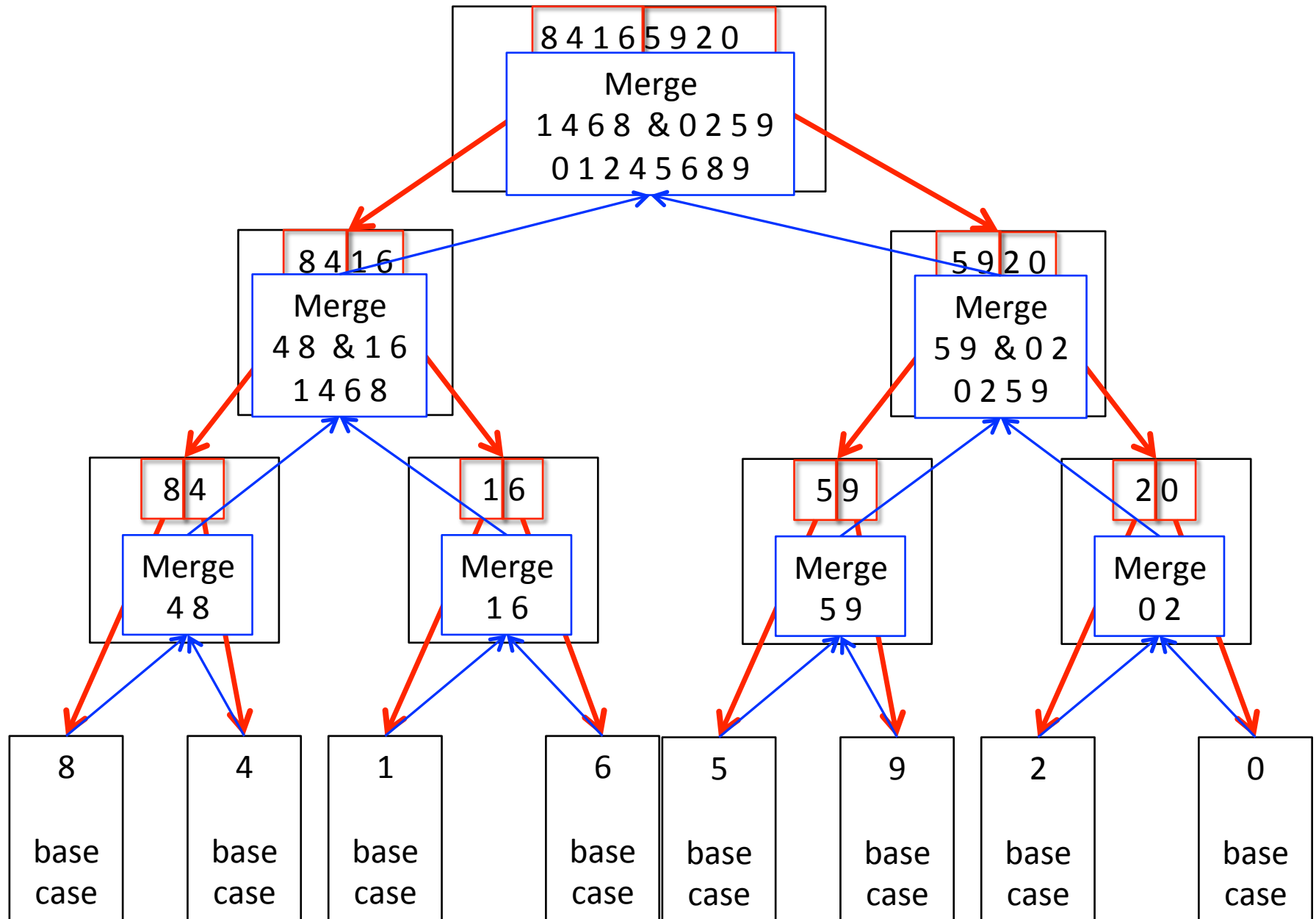
```
        return merge(left, right)
```

base case

divide

conquer with
the merge step

- **divide list** successively into halves
- depth-first such that **conquer smallest pieces down one branch** first before moving to larger pieces



COMPLEXITY OF MERGE SORT

- at **first recursion level**
 - $n/2$ elements in each list
 - $O(n) + O(n) = O(n)$ where n is $\text{len}(L)$
- at **second recursion level**
 - $n/4$ elements in each list
 - two merges $\rightarrow O(n)$ where n is $\text{len}(L)$
- each recursion level is $O(n)$ where n is $\text{len}(L)$
- **dividing list in half** with each recursive call
 - $O(\log(n))$ where n is $\text{len}(L)$
- overall complexity is **$O(n \log(n))$ where n is $\text{len}(L)$**

SORTING SUMMARY

-- n is $\text{len}(L)$

- bogo sort
 - randomness, unbounded $O()$
- bubble sort
 - $O(n^2)$
- selection sort
 - $O(n^2)$
 - guaranteed the first i elements were sorted
- merge sort
 - $O(n \log(n))$
- $O(n \log(n))$ is the fastest a sort can be

WHAT HAVE WE SEEN
IN 6.0001?

KEY TOPICS

- ✓ ■ represent knowledge with **data structures**
- ✓ ■ **iteration and recursion** as computational metaphors
- ✓ ■ **abstraction** of procedures and data types
- ✓ ■ **organize and modularize** systems using object classes and methods
- ✓ ■ different classes of **algorithms**, searching and sorting
- ✓ ■ **complexity** of algorithms

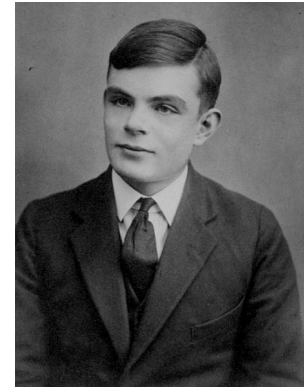
OVERVIEW OF COURSE

- ✓ ■ learn computational modes of thinking
- ✓ ■ begin to master the art of computational problem solving
- ✓ ■ make computers do what you want them to do

Hope we have started you down the path to being able to think and act like a computer scientist

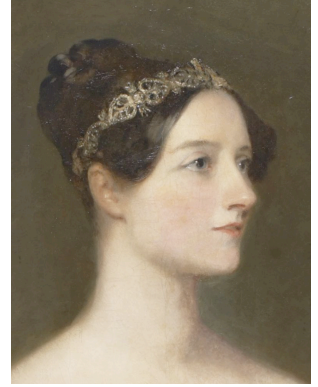
WHAT DO COMPUTER SCIENTISTS DO?

- they think computationally
 - abstractions, algorithms, automated execution
- just like the three r's: reading, 'riting, and 'rithmetic – computational thinking is becoming a fundamental skill that every well-educated person will need



Alan Turing

Image in the Public
Domain, courtesy of
[Wikipedia Commons](#).



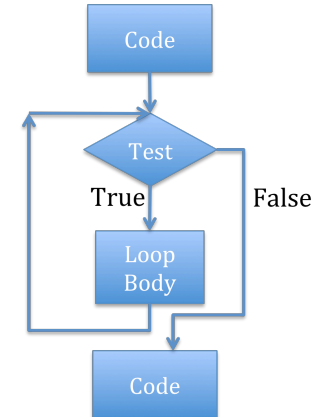
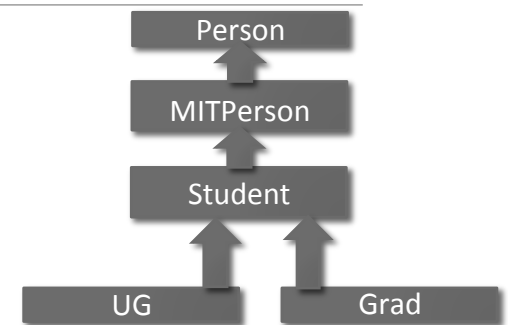
Ada Lovelace

Image in the Public
Domain, courtesy of
[Wikipedia Commons](#).



THE THREE A'S OF COMPUTATIONAL THINKING

- abstraction
 - choosing the right abstractions
 - operating in multiple layers of abstraction simultaneously
 - defining the relationships between the abstraction layers
- automation
 - think in terms of mechanizing our abstractions
 - mechanization is possible – because we have precise and exacting notations and models; and because there is some “machine” that can interpret our notations
- algorithms
 - language for describing automated processes
 - also allows abstraction of details
 - language for communicating ideas & processes



```
def mergeSort(L, compare = operator.lt):  
    if len(L) < 2:  
        return L[:]   
    else:  
        middle = int(len(L)/2)  
        left = mergeSort(L[:middle], compare)  
        right = mergeSort(L[middle:], compare)  
        return merge(left, right, compare)
```

ASPECTS OF COMPUTATIONAL THINKING

- how difficult is this problem and how best can I solve it?
 - theoretical computer science gives precise meaning to these and related questions and their answers
- thinking recursively
 - reformulating a seemingly difficult problem into one which we know how to solve
 - reduction, embedding, transformation, simulation

$O(\log n)$; $O(n)$;
 $O(n \log n)$;
 $O(n^2)$; $O(c^n)$

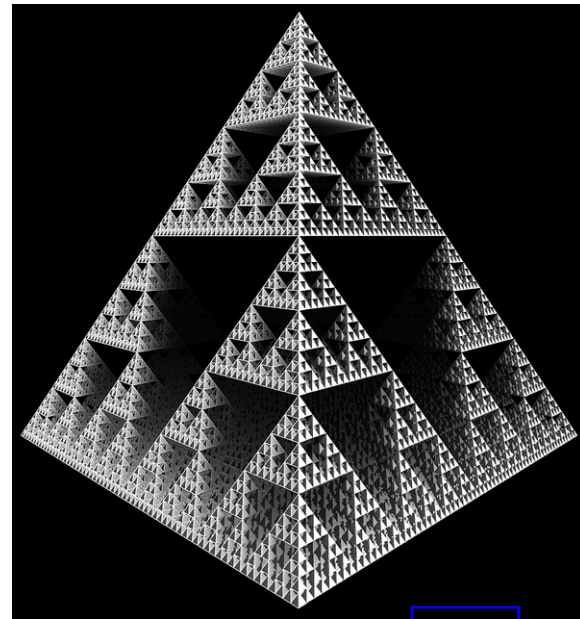


Image Licensed CC-BY, Courtesy of [Robson#](#) on Flickr.

MIT OpenCourseWare
<https://ocw.mit.edu>

6.0001 Introduction to Computer Science and Programming in Python
Fall 2016

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.